

Systematic Analysis of Ring-Oscillator-Based Side-Channel Countermeasures

The Good, the Bad, and the Noisy

Anonymous Submission

Abstract.

Hiding countermeasures play a crucial role in modern hardware security, offering a cost-effective complement to masking in order to protect against side-channel attacks. Within this category, noise generators are a common component used to artificially increase the system noise, thereby amplifying the effectiveness of masking. In many applications, noise generators are implemented ad hoc by running existing on-chip blocks in the background as a cheap but hard-to-control solution. Various academic works have proposed ring oscillators as an alternative, because of their compactness, controllability, and their capability to generate high switching activity at low area cost. Interestingly, while several works investigate ring oscillator configurations for side-channel protection, hardly any insights exist on how to systematically evaluate and profile ring oscillators in the context of a given application target.

In this paper, we aim to close this gap. We introduce a generalized approach to compare the security and cost of noise generators. It employs traditional side-channel evaluation methods (SNR, TVLA), measures traces-to-success under standard attacks, as well as robustness to filtering to assess the protection level. Implementation costs are measured in terms of area and power overhead. In a second step, we apply our methodology to analyze existing ring oscillator based noise generators on a Xilinx Artix-7 FPGA, before comparing them against a commonly deployed on-chip hardware module based on a full Keccak block with random input. Building on these results, we provide practical guidelines for designing efficient ring oscillator based noise generators that are tailored to specific applications, ultimately enabling practitioners to make informed choices when designing and deploying hiding countermeasures.

Keywords: Side-Channel Analysis · Hiding · Ring-Oscillator

1 Introduction

Power side-channel attacks exploit information leaked through power consumption: processing of intermediate secret-dependent values influences the power consumption and leaks information on secrets to an adversary with physical access. Based on measured power consumption traces, an adversary can then perform a variety of analyses that range from Simple Power Analysis (SPA), over more complex Correlation Power Analysis (CPA) to Deep Learning (DL)-based attacks. These attacks pose a threat to otherwise cryptographically secure algorithms; consequently a variety of countermeasure has been proposed and are widely deployed in real-world applications.

Power side-channel countermeasures can be broadly grouped into two categories: *Masking* breaks the dependence between the secret and intermediate values by splitting the secrets into multiple shares and performing computations on the shares separately. Thus, both the computed values and the side-channels are decoupled from the processing of intermediates. Masking provides provable security guarantees under theoretical models (e.g., [DDF14]), which makes it an attractive research target. However, this comes at a cost: Increasing the number of shares increases the data to be processed, leading to

significant time, or area overhead. Furthermore, non-linear operations require special handling and costly mask refreshes, which may demand high-quality good randomness sources. Moreover, to achieve masking security in real-world scenarios, a sufficient noise level is required [HFG⁺25].

In contrast, *hiding* aims at reducing the dependence of the side channel itself on the secret, so it typically does not provide provable security. Ideally, it requires the adversary to collect more measurements, which increases the computational cost of the attack. It is important to note that the dependency does not disappear, so the implementation remains vulnerable, albeit with greater effort required by an adversary. Potentially, for this reason, less academic research has been conducted in this area than in masking in recent years. Nevertheless, hiding is widely used in industry as a lightweight side-channel countermeasure, either alone or in combination with masking, because it is often cheaper to deploy: Examples include the secure element devices from NXP [Sem] and Samsung [SEC], which both incorporate hiding mechanisms, such as dummy operations and noise generation. Moreover, numerous patents from renowned companies claiming hiding countermeasures demonstrate the industry interest, starting with the differential power analysis countermeasures proposed by Kocher et al. [KJJ01]. More recent patents include hiding countermeasures from Qualcomm [PGX17], Intel [GDT⁺22] and IBM [CFK19] further underscoring the practical relevance of hiding countermeasures.

Mangard, Oswald and Popp state in [MOP07] that, for power side-channels, hiding can be realized via different means. One option is equalizing the consumed power, as in dual-rail logic [TAV02, TV04]. Another option is randomizing the power consumption, e.g., by generating additional noise [GM11]. This generated noise can be advantageous, especially when combined with masking. There are multiple ways to generate noise, ranging from specialized high power consuming circuits [NPBH23] to analog methods [HWL17]. However, as we will show in this work, each proposed solution employs its own evaluation method making it difficult to compare them to others. Consequently, it is unclear which technique is most efficient and secure for a given application and practical deployment is often unsystematic.

One promising class of noise generators is the utilization of Ring Oscillators (ROs) [LCL10, NYH13, MGZS17, KGS⁺19, PGC⁺22, NPBH23, Sch23, YKS⁺23], as they require little area, while inducing substantial switching activity, i.e., cause noise. In their simplest form, ROs consist of an odd number of inverters, and their oscillation frequency is determined primarily by the physical properties of those inverters. Unfortunately, research on the application of ROs as hiding countermeasures remains limited, largely because no unified evaluation framework exists to enable comparison across designs. Furthermore, the theoretical discussion of the required characteristics of the injected noise remains incomplete.

1.1 Our Contributions

In this paper, we present the first systematic comparison of RO-based noise generators. To this end, we first propose a generic structure for RO-based noise generators and use it to describe, unify and compare different designs from prior work. We then discuss the evaluation strategies used in these works and propose a unified evaluation methodology. In particular, we analyze the robustness to filtering that may be applied to the recorded traces. Furthermore, we build on the evaluation methods from prior work and analyze the measurements using methods based on average SNR, variance, CPA, and TVLA. All designs are implemented on an Artix-7 Field-Programmable Gate Array (FPGA) and evaluated against a statically routed, unprotected Advanced Encryption Standard (AES) implementation. We also compare the resulting numbers with those obtained from an independently running Keccak core that acts as a noise generator. Based on our results, we then derive practical guidelines for designing an effective RO-based noise generator tailored

to specific applications. Finally, we demonstrate the benefits of these guidelines by using them to design an RO-based noise generator tailored to our target system. This design improves upon prior designs evaluated in this work by an order of magnitude, showing that careful consideration of system parameters yield good results.

1.2 Related Work

The concept of generating noise to hide side-channel signals is as old as (power) side-channel attacks themselves. Kocher et al. [KJJ99] highlighted in their seminal work on differential power analysis that additional generated noise can make the attacks more difficult. In their accompanying patent [KJJ01], they describe the first noise generator to counter side-channel attacks. However, the research literature on concrete noise-generating engines remains sparse, as it is often argued that averaging additional measured traces can mitigate the effects of added noise [Sha00]. In current real-world devices, noise generation is often achieved by activating additional cores to perform independent computations, such as running another on-board block, e.g., a Keccak core fed with random data. These noise generation solutions are frequently implemented in an ad hoc manner, often without a thorough evaluation of their efficiency.

Most research papers in this area, however, focus on designing specialized devices to generate noise. [DMN⁺17] construct a noise generation circuit by using a Linear Feedback Shift Register (LFSR) that controls an additional current source. The two works [GM11, Gün12] analyze how the inherent building blocks of FPGAs can be utilized to generate noise in the power consumption independent of the protected implementation. In contrast, [KBG09, KBG11, TPZA25] propose protecting an AES-implementation by adding parallel computations on the data with a random key. The largest class of noise generators in side-channel research is RO-based, as these basic building blocks can be implemented in any hardware and promise high switching activity. Therefore, we evaluated and systematized different RO-based designs to derive a generic structure of RO-based noise generators, thus allowing for a precise and uniform description of the designs. In the next section, we will first describe this generic structure, before detailing the configuration options of the individual prior works.

2 Ring-Oscillator-Based Noise Generators

Ring Oscillators (ROs) are an important building block in electrical circuit design. In their simplest form, ROs consist of an odd number of inverters chained together, with the output of the last inverter feeding the first inverter. Because of the uneven number of inverters, the output of the final inverter is always different from the previous input of the first inverter. This causes the circuit to start oscillating. The speed of the oscillation, or frequency f_{ro} depends on the number of inverters n and the average delay time of a single inverter t_p [Grö07]:

$$f_{ro} = \frac{1}{2nt_p} \quad (1)$$

Longer chains lead to slower frequencies because the signal requires more time to propagate through the entire chain. This toggling of inverters, also referred to as switching, can be utilized to build a noise generator. Using a ROs as a noise generator is beneficial as they are structurally simple and universally applicable unlike solutions that rely on more specific building blocks, such as BRAMs. Note that any gate that is capable of inverting an input can also be used to build an RO. For example, designers can not only use inverters but also NAND gates with one of its two inputs tied high. In this work, we use Look-Up Tables (LUTs) to implement the inverting functionality because we evaluate the designs on a FPGA. In this section, we present the RO-based noise generators

from [LCL10, NYH13, MGZS17, KGS⁺19, PGC⁺22, NPBH23, Sch23, YKS⁺23]. To this end, we summarize the different designs and, based on this, propose our generic RO-based noise generator structure and the various configuration options. Our generic structure allows us to then present the individual designs precisely and in a uniform format. This overview of the designs provides an understanding of the current state of research and helps to explain the evaluation results of the designs in Section 4.

2.1 General Ring-Oscillator Architecture

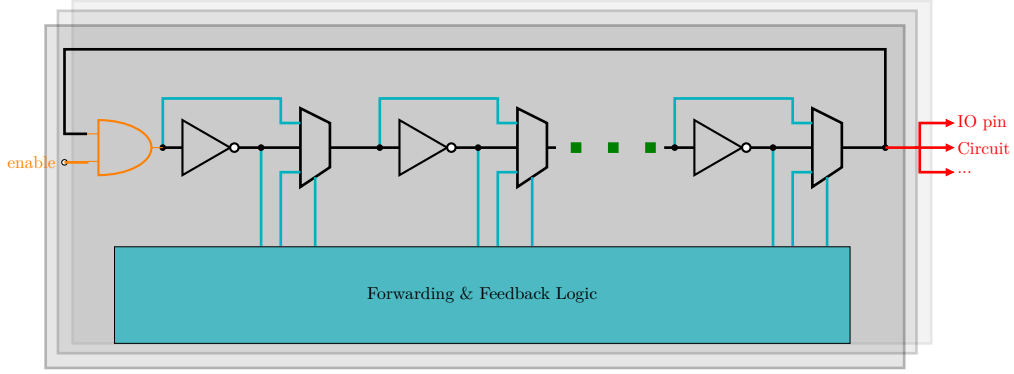


Figure 1: Our Generic Structure of Ring-Oscillator-Based Noise Generators.

Figure 1 illustrates our generic RO-based noise generator structure. The inverter chain is shown in black, whereas the configurable options are highlighted in different colors. The green dots indicate the adjustable chain length and the repetitive pattern in the background illustrates that multiple RO instance may be instantiated. The red arrows at the output represent different ways to connect the RO’s output to an external load. By fixing these parameters, a *static* RO can be defined. If the options can be changed dynamically during operation, we refer to it as a *dynamic* RO. To this end, the orange input logic can be used as it enables or disables the RO switching. The blue block indicates forwarding and feedback logic that enables dynamic reconfiguration of the RO design at runtime.

Table 1 shows the different RO-based noise generator designs and their configuration decisions. In the following, we elaborate on these different options in more detail, before we present the individual designs from prior work.

Number of Active RO Instances. The number of implemented ROs can be varied between one and an arbitrary amount that fits on the device. This increases the switching activity linearly with the number of instances. For dynamic ROs, an AND gate enables the activation and deactivation of the individual ROs. This can be used as a power-saving measure when the maximum amount of noise is not needed. That is because setting one input of an AND gate forces the output zero. If this output leads to an inverter stage, it will longer see a rapidly changing input signal and stabilize instead, thus stopping the oscillation. It can also be utilized to increase the generated noise if activations and deactivations happen so fast that most of the switching activity stems from the ROs settling into a steady state, as described in [Sch23].

Chain Length. By choosing the chain length of the RO, its oscillating frequency is fixed. This can be necessary if the generated noise targets a specific signal frequency. Dynamic ROs control the chain length dynamically by multiplexing the signal, so that one or

multiple inverters may be skipped. Thus, it is possible to adjust the oscillation frequency on-the-fly. The system designer can scale skipping structure to arbitrarily complex degrees, allowing him to build configurable blocks that are chained together and may themselves get skipped.

Randomization The input signal of dynamic ROs can be generated using different strategies: *Dependent* ROs derive the control signal from the secret computation that is happening in the protected module. This can lead to the generated noise depending on the secret, and thus, amplifying the signal. Depending on the exact logic, the leakage model of dependent ROs can change, so that attack strategies based on correct leakage models may fail. However, for attacks that can adapt to this leakage model, this noise generation would effectively result in an amplification of the secret, and, thus in a less secure system. For this reason, we are omitting these dependent ROs from our evaluations.

In contrast, the control logic of *independent* ROs is generated independently from the secret computation. This is typically realized using a Pseudorandom Number Generator (PRNG), which makes the behavior of the ROs is maximally unpredictable. Another possibility is to control the ROs based on the output of a (secret-independent) sensor or other signals. Thus, the noise generator can also be used as a fault injection monitor, as described in [YKS⁺23].

Amplification of Output. In order to take advantage of the high switching activity of the RO output signal, it can be further connected to other circuits. This amplifies the output signal. Circuits that can be driven by an RO output include IO pins (as seen in [KGS⁺19, YKS⁺23]), LFSRs, and other building blocks. The oscillating output signal can also be used as the clock input of such building blocks or existing on-board modules.

Cautionary Note: Placement. It is important to carefully consider where the ROs are placed in relation to the secret computing circuit. Seifoori, Mirzargar and Stojilovic describe in [SMS20] that if an RO is located close to gates or wires that transmit the signal, the secret can couple into the RO’s switching behavior and, thus, get amplified. For instance, Lundberg [Lun25] uses this effect by inserting an RO as a hardware trojan and exploiting the increased signal strength. Therefore, when using ROs as noise generators against power side-channel attacks, the ROs should be placed far away from secret computations.

Table 1: Configurations of the RO-Based Noise Generators from Previous Work

Countermeasure			Parameters		
			Chain Length	# of ROs	Output Amplification Randomization
Digital Controlled ROs	[LCL10, NYH13]		1	12 8	- RO-activation (dep. on random & input signal)
FiGaRO	[MGZS17]		>3	32	Outputs xored Linear Feedback RO
Active Fences	[KGS ⁺ 19]		1	372	Outputs xored to IO-Pin RO-activation by PRNG
Xored HCM	[PGC ⁺ 22]		1	8	Outputs xored RO-activation by PRNG
Configurable RO Chains	[NPBH23]		13 - 65	32	- Chain length randomization & RO-activation by PRNG
Random Number of ROs	[Sch23]		approx. 11	32	- RO-activation by PRNG
Programmable RO	[YKS ⁺ 23]		1 - 57	1	IO-Pin Chain length randomization by PRNG

2.2 Previous Designs

We now use the general structure introduced in the previous section to describe the noise generator designs from research literature. This enables the evaluation in Section 4 and a more detailed discussion. For each design, we provide the configuration choices, as well as the evaluation methodology used in the corresponding paper. Table 1 briefly summarizes the configurations, while Table 2 lists the evaluation methods used.

To the best of our knowledge, all prior works assess their noise generator designs by only performing single experiments. This means, that the resulting quantity (e.g., Signal-to-Noise Ratio (SNR) or Test Vector Leakage Assessment (TVLA) values), which is used to assess the effectiveness is only computed once for the amount of measured traces. This may cause the results to be influenced by factors other than the noise generation. Repeating the experiments and averaging the results provide a more reliable result. As Table 2 shows, the different papers mainly apply the TVLA and CPA to their measured data.

In a CPA, the attacker estimates the correlation coefficient between the possible values X and the measured distribution Y

$$r_{can} = \frac{\sum (x_i - \mu_x) \cdot (y_i - \mu_y)}{\sqrt{\sum (x_i - \mu_x)^2 \cdot \sum (y_i - \mu_y)^2}} \quad (2)$$

for all possible key(-byte) candidates with μ_x, μ_y being the mean values of X and Y . The key candidate with the maximal r_{can} wins is most likely to be the secret. In line with the prior work, we fix the success rate of the CPA at 100%. The quantities, provided by the papers, are either the number of traces for the CPA to reach this success rate, or the maximum value of r_{can} for the correct key guess.

The TVLA computes the t -statistics

$$t = \frac{\mu_r - \mu_f}{\sqrt{\frac{\sigma_r^2}{n_r} + \frac{\sigma_f^2}{n_f}}} \quad (3)$$

of two sets of traces (one with fixed data, one with random). A t -value larger than 4.5 typically indicates that the design is leaking side-channel information. These different evaluation strategies make it difficult to compare the designs directly. Therefore, we analyze these methods in Section 3 in more detail and discuss how to enable a fair comparison.

Digital Controlled ROs. The earliest approach, described by Liu, Chang and Lee [LCL10] and Nianhao, Zhou and Liu [NYH13], use twelve, respectively eight, one-stage ROs that are switched on and off depending on the input signal. In this context stage refers to a chain length of one. This input signal is an XOR combination of the AES S-box input and a random 8-bit number. This strategy was evaluated by simulating power consumption traces and performing a CPA on them. As discussed above, using the secret S-box input to control the ROs will likely alter the leakage function of the secret signal, leading to an incorrect correlation attack value. This assumption is supported by the results in [LCL10] showing that the correlation coefficients do not change for an increasing number of traces. Therefore, we directly feed the random numbers of a PRNG to the enable logic of the ROs for the evaluations in this work.

FiGaRO. Ma et al. [MGZS17] propose to feedback the RO output signal not only to the beginning of the chain but also to intermediate values in the RO, analogously to an LFSR. The design of the feedback net is based on Fibonacci and Galois LFSRs. The output of 32 Fibonacci and Galois RO instances is fed into an XOR gate. The authors evaluate this countermeasure using a CPA with 2,000 traces.

Active Fences. The noise generator of Krautter et al. [KGS⁺19] is designed to hide side-channel leakage on multi-tenant FPGAs. In this scenario, multiple users share a single FPGA, but utilize different areas. In order to prevent other areas from capturing the power consumption of a victim module, it is intentionally surrounded with single-stage ROs. The overall amount of ROs is roughly equivalent to the utilization of the design that should be protected, which is 672 single-stage ROs for their target system. Additionally, the enable signal of each RO is tied to an output of a random number generator, and all RO outputs are connected to an XOR gate, which in turn is connected to an IO-pin. For their evaluation, they perform a CPA and record the amount of traces with and without their countermeasure in place.

Xored High Consuming Module. In [PGC⁺22], Parrilla et al. advance their previously developed High Consuming Module (HCM) [PCT⁺15] and to the Xored HCM (XHCM). The HCM consists of four parallel single-stage ROs that drive a network of OR and AND gates. The XHCM replaces the AND/OR network with XOR gates, as they have a higher switching probability. The authors propose to instantiate the XHCM module with eight ROs and eight enable inputs, one for each RO, that are randomly activated by a True Random Number Generator (TRNG). This allows XHCM to generate randomized power amplitude spikes. Unlike other works, the authors do not perform a t-test or other side-channel related evaluations and base their security claim on measuring the dynamic power consumption of their solution, with a higher power consumption being better.

Configurable RO Chains. Nassar et al. [NPBH23] introduce configurable inverter elements to enable the dynamic reconfiguration of their design. It consists of multiple configurable RO-elements with different routing options. Many different configurations can be dynamically realized by combining single-stage elements with four-stage RO blocks, allowing for fine-grained frequency control. For evaluation purposes, they perform a t-test with 60,000 measurements.

Random Number of ROs. The RO-based design of Schaumont in [Sch23] can both act as a fault injection detection mechanism as well as a hiding countermeasure. The target design (a RISC-V core running at 4 MHz), is enhanced with 32 ROs that run at 120 MHz and are placed throughout the chip. The software dynamically activates a random number of ROs to generate broadband noise, meaning the noise spreads across multiple frequencies. This is in contrast to narrowband noise, which is localized at one specific frequency. Applying this countermeasure, a CPA using 50,000 traces does not succeed and a t-test with the same amount of traces showed no significant leakage.

Programmable RO. The programmable RO (PRO) of Yao et al. in [YKS⁺23] is not only designed for hiding, but also for power and fault injection monitoring. In order to prevent attackers from filtering out the noise, they also use dynamic ROs where the frequency is changed randomly during runtime. Thus, their example design can achieve frequencies between 22 MHz and 123.44 MHz. This so-called Programmable RO (PRO) instance is connected to an unused IO-pin to amplify its output. The effectiveness of the countermeasure is determined by both analyzing the frequency spectrum and performing a t-test with 50,000 traces.

Summary Previous work, as surveyed in this section, proposed numerous ways to configure RO based noise generators. However, the evaluation methodologies typically pick one type of empirical measure, and estimate this on one single experiment (see Table 2 for an overview). Reliable comparisons between configuration options are non-existent. Papers assume that attack strategies are not altered in an attempt to compensate for the effect of

Table 2: Overview of the Different Hiding Countermeasures with their Respective Evaluation Strategies.

Countermeasure		Metric	Evaluation Strategy		
			Method	Real-World Exp.?	# traces
Digital Controlled ROs	[LCL10, NYH13]	# traces max r_{can}	CPA CPA	- ✓	1,000,000
FiGaRO	[MGZS17]	max r_{can}	CPA	✓	2000
Active Fences	[KGS ⁺ 19]	# traces	CPA	✓	80,000
Xored High Consuming Module	[PGC ⁺ 22]	$\Delta W/LUTs$	power consumption	✓	-
Configurable RO Chains	[NPBH23]	t -value	TVLA	✓	120,000
Random Number of ROs	[Sch23]	t -value	TVLA	✓	50,000
Programmable RO	[YKS ⁺ 23]	t -value	TVLA	✓	50,000

the noise generator. We address this apparent gap in methodology in the next section, with the aim of not only comparing existing noise generator designs, but more importantly, to have a clear strategy to build cost-effective RO-based noise generators.

3 Noise Generator Evaluation

Designing an efficient noise generator requires a concrete evaluation methodology to ensure informed design decisions. We will discuss these in the following, before presenting the methodology used in this work.

3.1 Evaluation Method

The effect of additional noise is well understood in side-channel literature. Comprehensively explained in [MOP07], the term noise refers to the contribution in power consumption that is independent of the signal. Increasing the noise only, has a negative impact on the effectiveness of all types of side-channel attacks, including attacks using differential inputs, e.g. CPA. We can thus measure the effectiveness of noise generator designs by evaluating how the attack performance changes. This can, for instance, be done by fixing the success rate of the attack to a certain value, mostly 100%, and then determine the number of traces that are needed to reach it. However, as the summary of previous work in Table 2 indicates, other ways of measuring the effectiveness of noise generation have also been used. CPA attacks, such as many standard DPA attacks [MOS11], are “specific evaluation methods”, in the sense that they fix many assumptions (an assumed leakage model, an intermediate target value, a specific distinguisher). Consequently, if we wish to understand the impact of noise generation on many trace points, we would need to configure CPA attacks for all these trace points (rather than just the best target), which is extremely time consuming.

Less specific methods may overcome this problem. For instance, computing the SNR requires less assumptions already as it is not necessary to define a distinguisher, and one can make fewer assumption about the leakage model when defining the signal. The SNR is commonly defined by

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}} = \frac{\text{Var}(\text{Signal})}{\text{Var}(\text{Noise})} \quad (4)$$

. Activating a noise generator should lower the SNR as the signal power P_{signal} stays constant and the noise power P_{noise} increases. This measure is easy to estimate from

labeled traces and connects to attack difficulty, since lower SNR generally implies more traces are needed for an attack to succeed. However, still this still requires a reasonable leakage model, which in turn needs a sound understanding of the target implementation.

Finally, a non-specific detection test makes even fewer assumptions. The TVLA compares the distributions of two sets of traces under different inputs without a leakage model by computing

$$t = \frac{\mu_r - \mu_f}{\sqrt{\frac{\sigma_r^2}{n_r} + \frac{\sigma_f^2}{n_f}}}. \quad (5)$$

For additive noise, as generated by the noise generators considered in this work, the mean values μ_r and μ_f should not change when the countermeasure is activated. However, the variances of the distributions should increase for an active countermeasure. Hence, the effect of a noise generator cannot be observed directly by comparing t -values alone, but by analyzing how many traces (n_r , n_f) are required before the leakage threshold is exceeded.

Visualizing the sampled distributions of noise generators without considering any signal, also enables their comparison. Especially when the noise is Gaussian-distributed (as often assumed in side-channel research), the noise distributions can be compared by assessing the variance of the generated noise. In fact, the connection between the noise variance and the success rate, aka the required number of traces for a successful attack, is discussed in [dCGRP19]. There, the upper and lower bounds of the number of required traces is derived from the Mutual Information (MI) of the Additive White Gaussian Noise (AWGN) channel.

These four different evaluation strategies (CPA, SNR, TVLA and noise variance) have different limitations (e.g., due to assumptions on the leakage model or distribution), and therefore also different strengths in terms of interpretability and generality. Consequently, when comparing noise generators, they all should be considered. Another important aspect, especially when considering RO-based designs, is the frequency-dependency of the noise. We will discuss this effect and its implications in the following subsection.

3.2 Filtering

Power analysis attacks work because there is an exploitable relationship between the data dependent part of the overall power consumption (which is often called the signal) and some predictable secret value. Whilst many attacks are performed in the time domain, i.e. power values are sampled over time and the analysis uses these traces directly, they can also be performed in the frequency domain. When transforming power traces into the frequency domain, it is common that the signal is located in a relatively narrow part of the frequency spectrum (rather than it being spread evenly across the frequency domain). If a noise generator's contribution to the overall power consumption is in a part of the frequency spectrum that is different from the part where the signal is located, then, already simple signal processing techniques such as band pass filters, can effectively remove the added noise. Especially when using ROs, that switch on relatively fixed frequencies, to build noise generators, this fact needs to be considered.

We know from previous work that filtering can improve the success rate of side-channel attacks [PHF, MOP07]. This can be explained, for instance, with the SNR, as the ratio of the signal power and the noise power. By removing frequencies from the measurements that only contribute to the power of the noise, P_{noise} decreases, while P_{signal} stays constant. Hence, the SNR increases. This is also the case if the removed features contain signal, but more noise than signal. For some attack-strategies it might be beneficial to remove as much noise as possible, while other, more noise-tolerant methods, such as Soft Analytical Side-Channel Analysis (SASCA) and DL-based Side-Channel Analysis (SCA), benefit from more careful filtering. It is important to note that noise present in the same frequencies as

the signal cannot be filtered without loss of the SNR. Hence, an optimal noise generator should generate noise in the most dominant frequencies of the signal.

[SWL21] discussed this effect in detail and showed it in practice. They propose to identify the frequencies of the measured signal that have maximum SNR and only analyze them in the final attack. Therefore, the SNR is assessed for all measurable frequency bands individually. By choosing only the frequency band that contains the maximum signal strength and filtering out the other frequency spectrum, the SNR can be significantly increased. We detail further on that method in the next section.

3.3 Our Noise Generator Evaluation Method

For the evaluation of the presented RO-based noise generators, we use the above discussed measures on filtered data. For all evaluations requiring a signal, for instance the SNR, we instantiate an unprotected secret implementation which leakage model is understood. On this implementation, we first run the filter-analysis to find the maximizing bandpass filter for the SNR. Assessing the unprotected, but filtered measurements provides us with a noise baseline for the experiments.

For each noise generator, we then compute the SNR on the point of highest leakage. Further, we measure the number of traces that first lead to a t-value of $\geq |4.5|$ and the number of traces for a first order and 100% success rate of a CPA. We also compare the variances of the distributions. Importantly, we average the results of several (20) experiments to rule out any bias in the measurements, that might stem from data-dependent effects. Finally, to enable the comparison of the efficiency of the noise generation, we also provide the numbers of the area utilization and power consumption.

4 Evaluation

In the following section we first outline our measurement setup, and then present the results for the above designs using our previously described evaluation method.

4.1 Measurement Setup

In order to evaluate all designs, we use the Chipwhisperer CW305 board, which is designed for side-channel measurements. The FPGA is supplied with power directly using a lab power supply, thus bypassing the board's step-down converter, which would inject additional noise on the power rail. The power consumption is measured using an on-board shunt resistor. The output is then amplified and connected to an oscilloscope using a DC 50 Ω blocker, to remove voltage offsets as well as prevent ringing. Each statistical evaluation uses 100,000 traces that are captured using 2.5 GS/s. The mean power usage of a design is calculated by measuring the voltage on both sides of the shunt resistor. Due to measuring constraints the sample rate is lowered to 1.25 GS/s. The power usage is calculated over 500 traces and excludes any unrelated power consumption, e.g., the AES.

All ring oscillator designs are paired with an unprotected Google Vault AES [PG15], which is a fully parallelized round-based implementation with 16 S-boxes. One encryption requires 12 cycles, where the first two cycles handling resetting the internal state and the initial key addition respectively. An unprotected implementation allows us to easily analyze the impact of different hiding countermeasures as it provides a strong secret-dependent signal. A masked AES would require more traces and computational power to properly evaluate the impact of the countermeasure. However, if a countermeasure works particularly well to hide the unprotected implementation, it will hide the masked implementation just as well, as masking amplifies the noise of a system. For those designs that require random bits to toggle certain functionality, we include a small `xorshift32`

PRNG, which produces 32 pseudorandom bits per cycle. We use the standard AMD Vivado partial reconfiguration flow to define two partitions, one for the AES and one for the countermeasure, on our FPGA. These partitions are located far apart from each other, to avoid interference. Furthermore, we reduce spurious results which may be generated by changing the routing of the AES, by placing it in a static partition. Thus, the routing is fixed and all ring oscillator designs are hiding the exact same AES implementation.

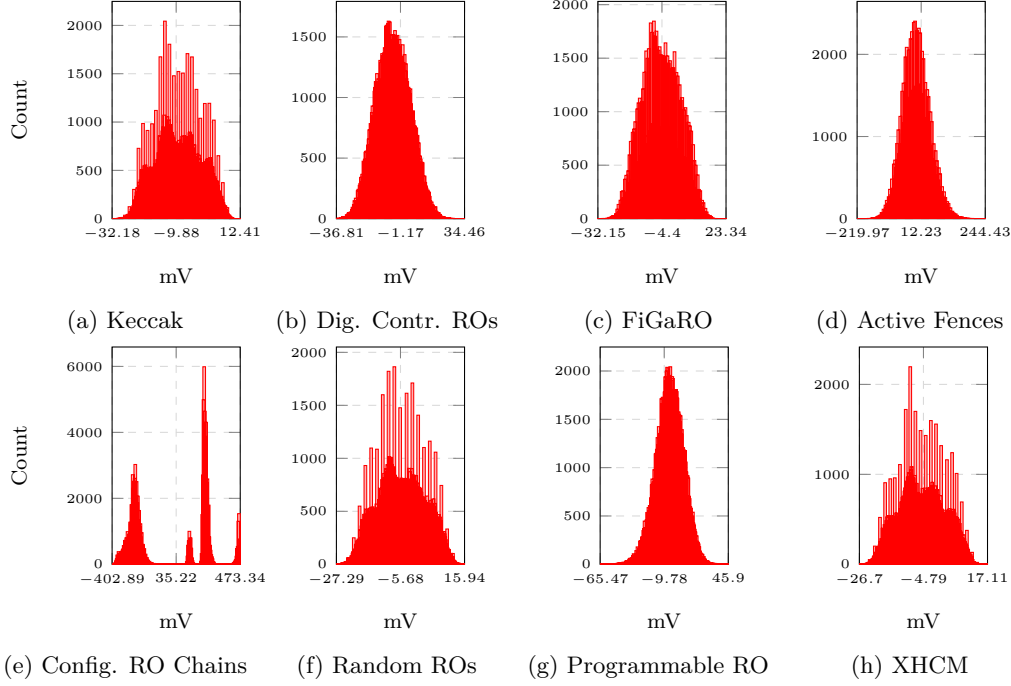


Figure 2: Noise distribution for all evaluated countermeasures.

Quantities. As detailed in [Subsection 3.3](#), we measure the variance of the generated noise to compare the effectiveness of the noise generators. To do so, we disable the AES core and record the power consumption of the design under test in isolation, acquiring 200,000 traces. We then sample all traces at the same time index to obtain the noise distribution. The exact sampling index can be chosen arbitrarily, because the noise generators are active continuously and should therefore yield the same distribution across the entire trace. While measuring the noise distribution, the AES always receives the same plaintext, thus eliminating any noise that could appear from processing random plaintexts. The resulting empirical distributions are shown in [Figure 2](#). Eventually, we compute the noise variance for each distribution.

In order to relate the variance to the concrete implementation, we measure the SNR of the unprotected AES and compare it to the SNR with the active countermeasures. We define the signal as the variance between mean power traces of different hamming distance groups during the last round, effectively modelling a last-round attack. This approach was selected because the chosen AES implementation forwards the shifted S-box output directly to the result registers. In order to rule clearly identify the effect of different RO designs, the SNR is performed on semi-fixed plaintexts, where only the first key byte is randomly chosen and all other bytes fixed to 0. This eliminates noise that could stem from processing completely random plaintexts. To rule out outliers, we repeat this experiment

20 times and average the resulting SNRs.

Furthermore, to connect our experiments to the evaluation methods done by the authors of the tested designs, we analyze the Measurements to Disclosure (MTD) of TVLA and CPA. For TVLA, we estimate the number of traces required to exceed the detection threshold and indicate leakage. To this end, we measure two sets of traces with the same principle used for the SNR: First, a fixed set, in which the plaintext is held constant, and, second, the semi-fixed set, in which all plaintext bytes are fixed to 0 except for the first byte, which is chosen randomly. Like in the SNR experiments, we run the TVLA evaluation 20 times to rule out outliers due to data specific leakage. Additionally, we only analyze the TVLA at the highest SNR point, because we are not interested in leakage points caused by other effects during earlier rounds.

The traces we record for the TVLA are also used for CPA experiments. As only the first byte is random, the information available to the CPA is severely limited. We therefore ran several experiments on the unprotected design to identify the best performing bytes. Out of the best performing bytes we chose byte 11 of the last S-box for our evaluation. However, it would have been possible to choose any other good-performing byte. The results of 20 evaluation CPA runs are averaged and evaluated. In this work, we record the absolute rank of the correct key byte and not the correlation coefficient.

Lastly, we specify the resource consumption of the countermeasures in terms of area, specifically as LUTs and slices on an FPGA, and power consumption.

Filter To ensure that the designs generate noise within the correct frequency range and that the generated noise cannot simply be filtered out, we filter all measurement data before performing the analysis discussed above. For this purpose, we use the method by [SWL21] as described in Section 3 and analyze the SNR of the band-filtered, unprotected AES. In Figure 3, we show the maximum SNR for different frequency bands of the unprotected AES implementation after applying different 10MHz bandwidth bandpass filters starting at 5 MHz. For the low end of the spectrum we used a 5 MHz lowpass filter, as a bandfilter would not apply. The SNR is maximized for the frequency band between 15 and 25 MHz. Therefore, we filter all measured traces with such a bandpass-filter.

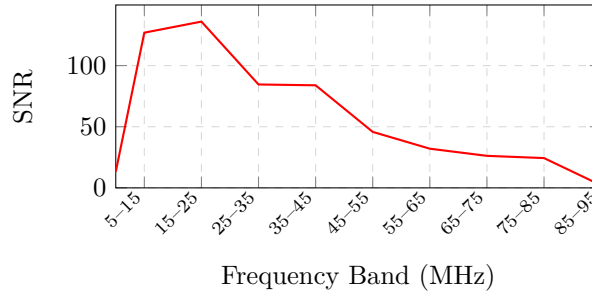


Figure 3: SNR of an unprotected AES without countermeasures by frequency band.

The effectiveness of applying filters to the generated noise can be seen in Figure 4, which depicts the frequency spectrum of an unprotected AES and of an AES that uses active fences. Since active fences toggle large logic blocks frequently, it creates most of the noise at lower frequencies as these blocks slowly start up after activating their enable signal. However, the AES is running at 10MHz, so all logic and intermediate gates must run at least at 10MHz. Note that the switching activity after a clock edge can be much higher than 10 MHz as the input propagates through all gates. As pointed out earlier, the maximum SNR is located in the frequency band between 15 and 25MHz. Thus, as shown in the graph, most of the low frequency noise gets filtered out. In the case of active fences, there is still residual noise in the frequency band that we analyze, but most of the power

that is used to generate noise is effectively wasted. Figure 5 shows that before filtering, a large amount of noise with high variance was generated, but after applying the bandpass filter, only a tiny fraction of this noise is left.

Given these strong effects of filters on the resulting quantities, it is highly required that noise generators are evaluated under appropriate filtering. These filters need to be tailored to the target implementation and system, e.g., following the strategy discussed above by [SWL21]. Consequently, we apply a 15-25MHz bandpass filter to our measured traces for all subsequent experiments. Thus, we yield the results in Table 3, which are discussed next.

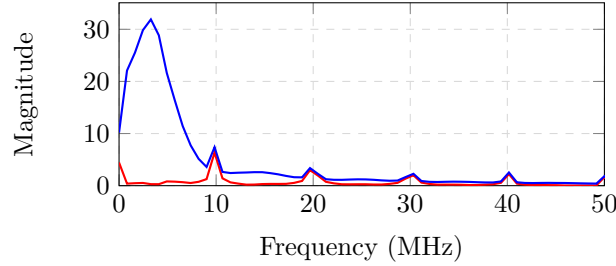


Figure 4: FFT of an unprotected AES (red) and an AES employing active fences (blue).

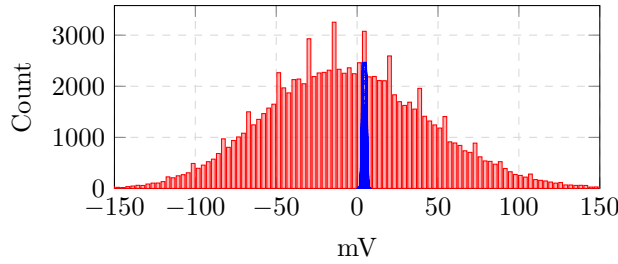


Figure 5: Noise distribution of active fences without filtering (red) and with filtering (blue).

4.2 Results

We conduct all experiments using the above presented setup and same conditions. The implementation of the noise generators are based on the information provided by prior work, summarized in Table 1. This includes possible duplicate instantiations in order to match the amount of S-boxes in our target design or the overall area taken by our chosen AES implementation. If prior work specified secret-dependent values as the input to an design, we replaced it with a pseudorandom values generated by the PRNG described earlier. Lastly, in order to compare RO countermeasures to noise that might be generated by simply running on-board blocks in parallel, we implemented a Keccak core and evaluated the generated noise as well. The results are shown in Table 3.

Our results show that in general the four quantities approximately proportional to each other. An increase in the noise variance generally correlates with a decrease of the SNR and an increase in the number of required traces for TVLA and CPA. This is illustrated by Figure 6, where the (averaged) number of traces for a successful CPA is plotted against the noise variance. The overall trend of an increasing variance correlating with a higher number of traces can be clearly observed, with a few outliers. For comparability with SNR numbers reported in other side-channel literature, we note that our SNR results are that high primarily due to of the filtering. The unfiltered SNR values, especially that for the

Table 3: Evaluation Results of the Previously Described Proposals. Byte 11 for CPA

Countermeasure	SNR	Results			Area		Power mW
		Noise Variance	# Traces for TVLA	# Traces for CPA	LUTs	Slices	
Unprotected AES	135.98	0.005	10	825	1414	415	87.12
Digital Controlled ROs [LCL10, NYH13]	122.65	0.009	21	1120	446	142	27.92
FiGaRO [MGZS17]	8.58	0.247	1110	8570	656	192	138.83
Active Fences [KGS ⁺ 19]	8.39	0.277	517	11920	941	314	264.4
Xored High Consuming Module [PGC ⁺ 22]	138.52	0.004	19	4775	22	7	3.48
Configurable RO chains [NPBH23]	0.65	3.701	6750	6960	4027	1146	213.2
Random Number of ROs [Sch23]	139.03	0.005	19	740	329	100	5.49
Programmable RO [YKS ⁺ 23]	93.26	0.011	33	2490	45	17	3.87
Keccak	320.33	0.020	171	2450	1077	287	9.17

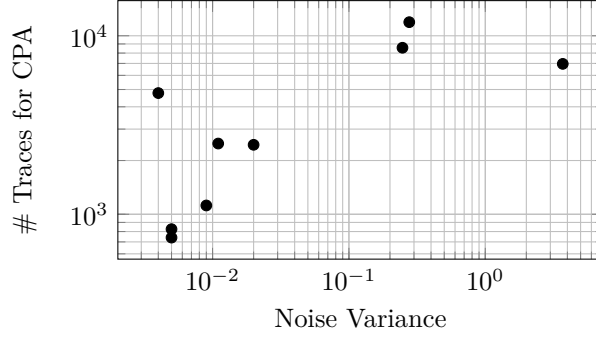


Figure 6: Influence of noise variance on the amount of required CPA traces.

unprotected AES implementation, are two orders of magnitude lower than the filtered values.

An outlier of the correlation trend is the SNR value of the Keccak core. Although the noise variance is than in the unprotected AES implementation, the SNR increases by more than a factor of two. This suggests an increased signal strength. In contrast to the other noise generator designs in this evaluation, the Keccak core shares the clock distribution network with the secret AES computation, which may lead to an amplification of the secret-dependent consumed power. This could be an explanation, but we were not able to identify the exact cause of this effect.

More importantly, these findings show that assessing only one of these quantities, even when averaging over multiple experiments, is not sufficient. With our methodology, we obtain a comprehensive picture of the efficacy of the different noise generation capabilities. This allows us to compare the efficiency of the noise generators. Nevertheless, it does not provide precise security predictions for the designs. To assess exactly how a noise generator design affects the system security, it is necessary to perform the strongest possible attack including optimal filtering, for the specific target implementation and system and evaluate its impact on the success rate. To assess exactly how a noise generator design affects the system security, it is necessary to perform the strongest possible attack for the specific target implementation and system. This implies that the attack uses optimal filtering and leakage models. By evaluating the impact on the success rate of this best-case attack, the effect of the noise generator can be identified. In the following, we use the comparative capability of our method to compare the different designs in terms of security, resource utilization, and efficiency.

Comparison. It strongly depends on the requirements and constraints of the target system how the trade-off between security enhancement of the noise generator and its resource

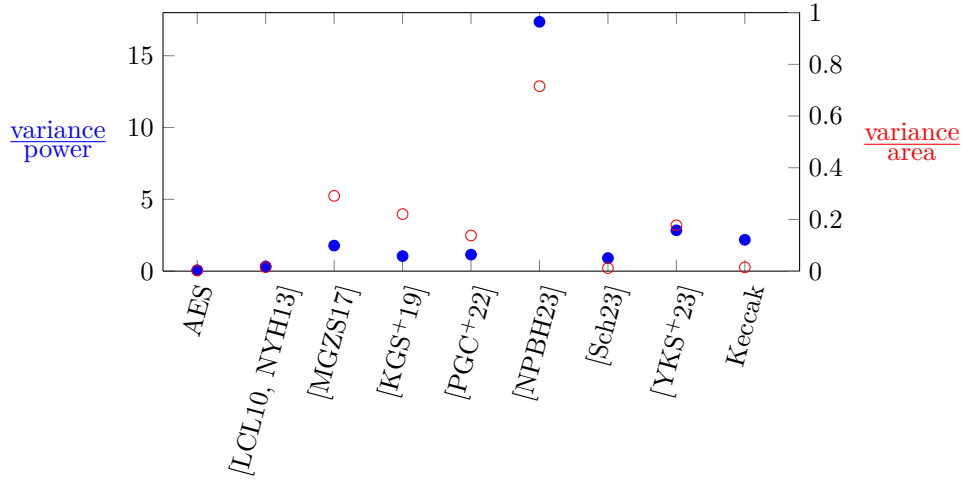


Figure 7: Efficacy of RO-based noise generators

utilization is performed. In this work, we provide a rough estimate of this trade-off by computing the ratio of generated noise variance to area and to power. The resulting efficiency ratios are shown in Figure 7 together with the baseline of the noise efficacy of the unprotected AES.

It can be seen that the efficiency of the Digital Controlled ROs [LCL10, NYH13], the Xored High Consuming Module [PGC+22] and the random number of ROs [Sch23], lie around a common baseline. All of these noise generators instantiate a certain amount of single-stage ROs that are randomly activated. This seems to limit their hiding effectiveness.

The Programmable RO [YKS+23] also shows no significant impact on the system noise level, but its very small area and low power consumption lead to a higher efficiency than the single-stage ROs. This effect might be explained by the noise amplification due to connecting the output to an IO-pin and dynamically varying the chain length. The impacts of these two factors on the security of hiding countermeasures are analyzed in detail in the next section.

FiGaRO [MGZS17] and Active Fences [KGS+19] are roughly an order of magnitude larger in area than the previously considered solutions. Nevertheless, this pays off as the noise variance also increases by about an order of magnitude. However, the power consumption is roughly two orders of magnitude higher than that of the unprotected AES. As shown in Section 4.1 for active fences, the generated noise spectrum is broader-band than the secret signal and is therefore less efficient in the target band.

Configurable RO chains [NPBH23] again increase the noise variance by about an order of magnitude. However its size is approximately ten times larger than FiGaRO and Active Fences and approximately the same size of the AES. The power consumption is lower than that of active fences, better matching the secret frequency band. However, the power consumption remains roughly twice that of the unprotected AES, therefore this design is also far from efficient.

By comparing different designs, we can determine which design choices lead to more efficient noise generators. However, this evaluation does not explain the specific effects of the configuration options. If the target application has constraints that none of the designs meet, the strengths of the different designs need to be combined. In order to enable the design of more powerful noise generators, we provide guidelines for it in the next section.

5 Guideline

The different configuration options of RO-based countermeasures, which we describe in Section 2 affect the efficiency of the noise generator in different ways. In this section, we analyze these effects separately, before aggregating them to develop tailored noise generators for our setup. For the sake of readability, we only provide the numbers of the generated noise variance for the results in this section.

Chain Length. Recall Equation 1 and that increasing the number of inverters n in the RO chain causes the output frequency to scale inversely with n (i.e., by $\frac{1}{n}$). We measure the output frequency by routing the RO output directly to an IO pin of the board. We then capture this signal and estimate its frequency. Figure 8 shows the measured output frequency as a function of RO chain length.

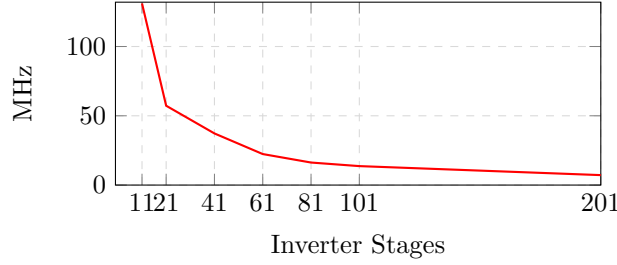


Figure 8: Output frequency of a RO measured through a IO pin.

It must be noted that the per-inverter switching frequency is set by the inverter delay t_p and does not change with the number of stages. If the RO output does not drive an external load, the dynamic power is dominated by the internal switching of the inverters. Thus, the dominant component in the power spectrum should be the stage switching frequency of a single-stage RO, given by $f_1 = \frac{1}{2t_p}$, no matter how long the chain length is. Equivalently, n stages produce n interleaved current spikes per period $T_{ro} = \frac{1}{f_{ro}}$, yielding an aggregated frequency at f_1 , which is independent of n . In our case, $f_1 \approx 1.35$ GHz which is beyond the bandwidth of our laboratory instruments. If our lab equipment could measure it, the parasitic capacitances of board and probe would act as a low-pass filter, strongly attenuating this spectral component making it unobservable.

Number of Ring-Oscillators. This effect also appears in our experiments that investigate the impact of increasing the number of ROs. The common assumption in related work, which we also adopt, is that the noise variance increases linearly with the number of ROs. However, neither measurements with varying numbers of 57-stage ROs ($f_{ro} \approx 23.6$ MHz) nor those with 71-stage ROs ($f_{ro} \approx 18.9$ MHz) show any change in the resulting noise variance. Enabling or disabling filtering does not change the results. Also the fact whether filtering of the data is enabled or disabled does not change the result. This further indicates that, without driving an external load, the noise is so high-frequency that it has no observable effect on the attack measurements.

A different picture emerges when instantiating varying numbers of ROs with randomized chain lengths such as the programmable RO countermeasure. It contains between 1 and 56 stages, changing the frequency every cycles, while not driving an external load. Figure 9 shows these results. They indicate that, if a single active RO instance has impact on the noise variance, increasing the number of instances results in a linear increase of the variance.

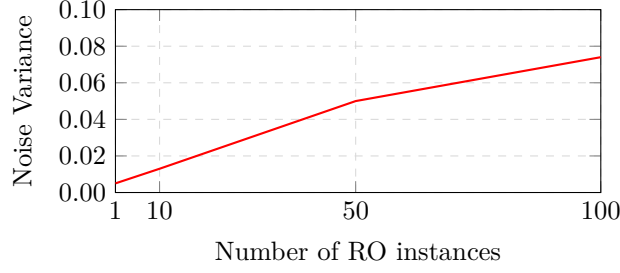


Figure 9: Noise variance of varying number of randomized RO instances.

Table 4: Evaluation results of different external load experiments.

RO Type	Output Driver	Variance	Area		Power mW
			LUTs	Slice	
Static RO 71 Stages	-	0.0040	50	14	0.62
Static RO 71 Stages	IO Pin	0.0180	68	19	2.31
Static RO 71 Stages	Keccak	0.1340	1150	307	105.59
Static RO 71 Stages	LFSR32 Clock (SRL)	0.0047	146	43	1.89
Static RO 71 Stages	LFSR96 Clock (SRL)	0.0049	147	47	1.95
Static RO 71 Stages	LFSR96 Data (SRL)	0.0040	147	47	1.45
Static RO 71 Stages	LFSR1024 Clock (SRL)	0.0053	176	51	2.01
Programmable RO	-	0.0054	45	15	1.89
Programmable RO	IO Pin	0.0112	45	17	3.87

Amplifying the Output. In order to amplify the output signal of an RO, its output can drive another circuit. Thus, toggling at the RO output induces additional switching activity, which effectively amplifies the noise power consumption. The external loads used in the evaluated designs either employ `xor` logic circuits as in [MGZS17, KGS⁺19, PGC⁺22] or an IO-pin as in [KGS⁺19, YKS⁺23] as the external load.

To achieve an amplifying effect in the power consumption, the load must draw high current. This can be achieved by connecting high-current-drawing elements such as an IO-pin. We evaluated the effect of connecting an IO-Pin to the RO output by measuring the programmable RO design of [YKS⁺23] once without and once with the IO-pin connected. The results are shown in Table 4: Driving the IO-pin approximately doubles the noise variance in our setup. The overall power consumption of the noise generator increases roughly by a factor of two, as it now drives the relatively large IO net. Similarly, when connecting an IO pin to a static 71-stage RO, the variance and power usage increases.

The high current drawing effect can also be achieved by driving many parallel switching gates with the RO. This can be done by connecting the RO output to a large circuit. To evaluate this, we first connect the RO to the input of a 96-stage LFSR. However, the results of this experiment, also shown in Table 4, show no measurable impact on the noise level. This is because in this configuration, the RO only drives the first input stage of the LFSR. The remaining stages of the LFSR are switching at the system clock frequency.

A large input net that can be driven by the RO output is, for instance, the clock distribution network of a digital circuit. Instead of connecting the RO output to the data input of the LFSR, we connect it to its clock input. This causes approximately half of the LFSR registers to switch each time the RO output toggles. However, our experiments showed that the physical realization of the LFSR has a strong impact on the efficiency of the noise generator. As Table 4 indicates, an RO-driven LFSR built from Shift-Register LUTs (SRLs) (a common FPGA building block) showed only a small increase in the noise variance, while the same configuration built with standard registers increased the variance significantly.

If a Keccak module or other on-board block is available, routing the RO output to its clock input yields even better results than the driving an LFSR. This is likely because the

combinational logic between the Keccak registers further increases its power consumption. However, as seen in the table, the power consumption rises significantly. As the RO output is not a proper clock signal, but instead a normal signal like any other gate output, it does not satisfy the usual requirements for a clock signal. In particular, routing one signal to hundreds of flip-flops introduces skew, which in term leads to glitching.

We conclude that connecting an external load to the RO output leads to an amplification of RO-generated noise. The larger the input net driven by the RO, the higher the amplification. Note that large circuits alone are not sufficient, as it is primarily the size and capacitance of the driven input net that determine the increase in noise power.

Dynamic ROs. As discussed above, dynamic ROs can be realized by either dynamically activating full RO instances, or by dynamically changing the chain length. A small PRNG, described in Section 4, is used to generate the necessary randomcontrol signals. We evaluate the effects created by dynamic ROs by analyzing the resulting frequency spectrum of the Active Fences countermeasure [KGS⁺19] in Figure 10 and compare it the spectrum generated by the Programmable RO countermeasure [YKS⁺23]. Active fences use static ring oscillator blocks and toggle a random amount on and off to generate noise. This in turn creates low frequency broadband noise as seen in the FFT spectrum, because the blocks slowly start drawing power as the ring oscillators start up. However, no significant noise is generated in other frequencies and the broadband noise becomes less powerful in higher frequency bands. This is especially noticeable when calculating the difference between an unprotected AES spectrum and spectrum of a AES protected by active fences. The frequencies clearly differ below 10 MHz, but equalize after 30 MHz. In contrast to this, programmable ROs only create minor low frequency noise, which is probably caused by activating chains within the oscillator or by the relatively large IO net. Calculating the FFT difference reveals that the design generates noise over a broad spectrum above 40 MHz. This is consistent with the the amount of stages which can dynamically range between 1 and 57 stages.

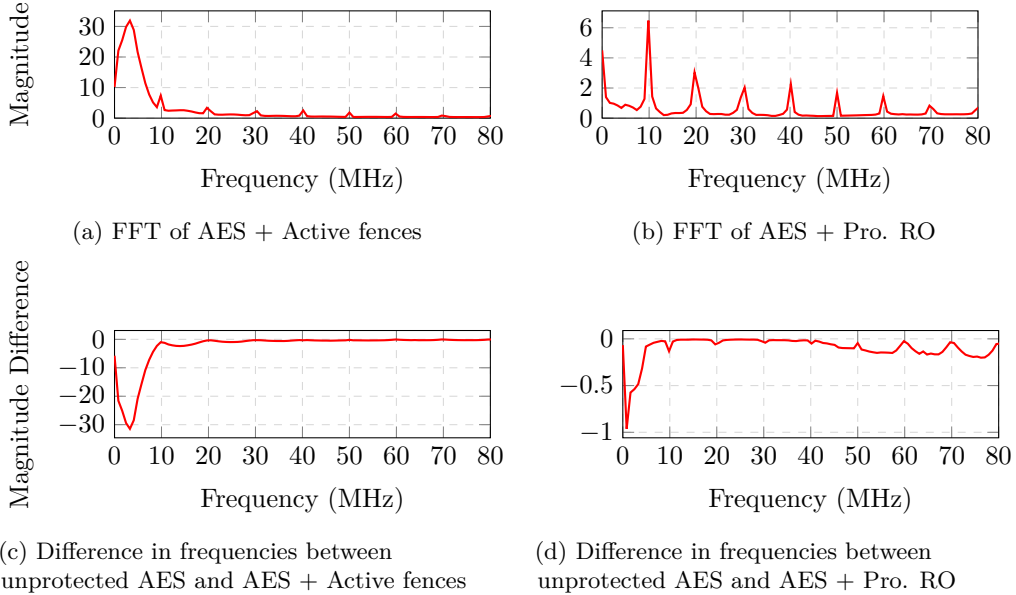


Figure 10: Effects of enable signal randomization (active fences) vs. chain length randomization (programmable RO).

Additionally, the designer can also decide how often the PRNG updates the RO

configuration. We assess the effects of this by performing different experiments with the two countermeasures with different PRNG update cycles and yield Figure 11. With increasing randomization periods, the overall noise variance steadily decrease. Therefore, designers should aim to randomize the RO configuration as often as possible. Furthermore, power consumption measurements showed that less regular configuration updates do not have significant influence on the power consumption. This intuitively makes sense, as under the assumption of ideal randomness, active fences will always have around 50% of all RO blocks activated. For programmable ROs less regular updates mean less chain length modifications, meaning the RO will stabilize at a set frequency, which lowers the variance.

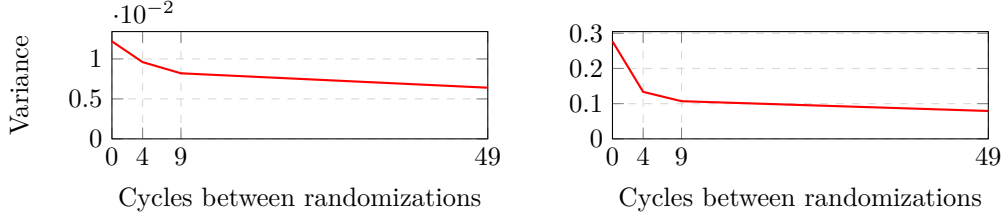


Figure 11: Influence of randomization periods on the noise variance.

5.1 Building a Tailored RO-Based Noise Generator

Table 5: Evaluation Results of Different External Load Experiments.

Output Driver	Variance	Area		Power mW
		LUTs	Slice	
Keccak	54.41	1230	338	37.45
LFSR1024 x16 (SRL)	0.088	711	199	3.89
LFSR1024 x4 (FDRE)	2.023	147	873	18.12
LFSR1024 x1 (FDRE)	0.137	141	247	3.45
LFSR512 x1 (FDRE)	0.041	147	138	2.49

Based on these guidelines, we design an RO-based noise generator tailored to our specific system parameters. As previously noted, the target AES core operates at 10 MHz, with the primary leakage occurring in the 15 to 25 MHz frequency band. Therefore, we configure a dynamic RO to operate within this specific range. The proposed design builds upon the programmable RO countermeasure, offering a compact footprint while enabling dynamic frequency switching. We instantiate an inverter chain with 57 stages, setting the base RO frequency at the upper limit of the target band. To achieve the lower end of the frequency band, we integrate additional delay stages (two 2-stage, two 4-stage, and two 8-stage chains), allowing the RO length to be expanded up to 85 stages. For noise amplification, we evaluated multiple approaches as seen in Table 5. If a Keccak module is available on the device, routing the RO output to its clock input proves highly effective. This configuration yields a noise variance of 54.41, which is an order of magnitude improvement over the baseline results presented in Section 4. Unlike the 71-stage RO from earlier, this design only draws 37 mW. This is probably caused by additional structures, like multiplexers, that are placed inside the RO. These may have better driving properties than typical LUTs, leading to less skew and less glitching. Alternatively, we consider driving a long LFSR that utilizes registers instead of SRLs, as SRLs create inferior results. Specifically, driving a single 512-bit LFSR produced a variance of 0.041, incurring only a moderate hardware overhead. Scaling the LFSR approach improved the variance to 0.13 for a single 1024-bit LFSR, and up to 2.023 when deploying four parallel 1024-bit LFSRs.

5.2 Open Questions

In this paper, we focus on ROs under normal operating conditions and restrict their placement to prevent inadvertent coupling of secret-dependent data into hiding countermeasures. However, under an expanded adversarial threat model, both operating conditions and placement strategies significantly impact the effectiveness of the RO-based noise generator. Assuming an adversary capable of accurately recording Electromagnetic (EM) emissions, placing a noise-generation block in close proximity to the hiding target becomes necessary. Conversely, as established earlier, the RO itself must remain physically isolated from the target. Future work should investigate how these competing placement requirements influence the overall efficiency of hiding countermeasures. In particular, an evaluation on an Application-Specific Integrated Circuit (ASIC) could yield valuable insights, as its multi-layered architecture offers three-dimensional spatial flexibility, contrasting with the strictly two-dimensional routing plane of an FPGA.

Furthermore, the device under attack may not consistently operate at room temperature. An adversary could intentionally heat or cool the device to manipulate RO frequencies. This, in turn, alters inverter delays due to the temperature-dependent properties of silicon. Similar effects can be achieved through intentional over- or undervolting. Consequently, future research should explore how ROs can be augmented to detect and actively respond to such abnormal environmental conditions.

6 Conclusion

Power side-channel security remains a critical issue for modern embedded devices. Noise generation is a possible countermeasure that increases the system noise level and, thus, forces potential attackers to take more measurements, increasing the overall cost of an attack. In this work, we analyzed state-of-the-art RO-based noise generators. We found that previous works used different evaluation methods, making it difficult to compare their security and efficiency. Additionally, certain parameters such as the frequency spectra of the target and the noise generators were not taken into account, allowing possible adversaries to filter out the generated noise. To address this, we evaluated prior designs under unified conditions, with filtering applied and assessing the four quantities SNR, noise variance, and number of traces for TVLA and CPA. Building on these findings, we provided design guidelines to build more effective and tailored RO-based noise generators. Applying these principles to our own target, we were able to build countermeasures that are two orders of magnitude more effective and efficient than comparable designs from prior work.

References

- [CFK19] Kangguo Cheng, Shawn P. Fetterolf, and Ali Khakifirooz. Hardware based cryptographic side-channel attack prevention, August 2019.
- [dCGRP19] Eloi de Chérisey, Sylvain Guilley, Olivier Rioul, and Pablo Piantanida. Best information is most successful. *IACR TCHES*, 2019(2):49–79, 2019.
- [DDF14] Alexandre Duc, Stefan Dziembowski, and Sebastian Faust. Unifying leakage models: From probing attacks to noisy leakage. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 423–440. Springer, Berlin, Heidelberg, May 2014.
- [DMN⁺17] Debayan Das, Shovan Maity, Saad Bin Nasir, Santosh Ghosh, Arijit Raychowdhury, and Shreyas Sen. High efficiency power side-channel attack immunity

- 737 using noise injection in attenuated signature domain. In *IEEE HOST 2017*,
738 pages 62–67. IEEE Computer Society, 2017.
- 739 [GDT⁺22] Santosh Ghosh, Debayan Das, Carlos Tokunaga, Avinash L. Varna, and
740 Joseph FRIEL. Countermeasures against hardware side-channel attacks on
741 cryptographic operations, January 2022.
- 742 [GM11] Tim Güneysu and Amir Moradi. Generic side-channel countermeasures
743 for reconfigurable devices. In Bart Preneel and Tsuyoshi Takagi, editors,
744 *CHES 2011*, volume 6917 of *LNCS*, pages 33–48. Springer, Berlin, Heidelberg,
745 September / October 2011.
- 746 [Grö07] Markus Grözing. Untersuchung des rauschens in komplementären metall-oxid-
747 halbleiter-ringoszillatoren. 2007.
- 748 [Gün12] Tim Güneysu. Using data contention in dual-ported memories for security
749 applications. *J. Signal Process. Syst.*, 67(1):15–29, 2012.
- 750 [HFG⁺25] Dina Hesse, Jakob Feldtkeller, Tim Güneysu, Julius Hermelink, Georg Land,
751 Markus Krausz, and Jan Richter-Brockmann. t-probing (in-)security - pitfalls
752 on noise assumptions. Cryptology ePrint Archive, Report 2025/1202, 2025.
- 753 [HWL17] Minghui Han, Megan Wachs, and Hai Lan. Countermeasure to power anal-
754 ysis attacks through time-varying impedance of power delivery networks,
755 September 2017.
- 756 [KBG09] Najeh Kamoun, Lilian Bossuet, and Adel Ghazel. Correlated power noise
757 generator as a low cost dpa countermeasures to secure hardware aes cipher.
758 In *SCS 2009*, pages 1–6. IEEE, 2009.
- 759 [KBG11] Najeh Kamoun, Lilian Bossuet, and Adel Ghazel. A masked correlated power
760 noise generator use as a second order dpa countermeasure to secure hardware
761 aes cipher. In *ICM 2011 proceeding*, pages 1–5. IEEE, 2011.
- 762 [KGS⁺19] Jonas Krautter, Dennis R. E. Gnad, Falk Schellenberg, Amir Moradi, and
763 Mehdi Baradaran Tahoori. Active fences against voltage-based side channels
764 in multi-tenant fpgas. In David Z. Pan, editor, *Proceedings of the International
765 Conference on Computer-Aided Design, ICCAD 2019, Westminster, CO, USA,
766 November 4-7, 2019*, pages 1–8. ACM, 2019.
- 767 [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis.
768 In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages
769 388–397. Springer, Berlin, Heidelberg, August 1999.
- 770 [KJJ01] Paul C. Kocher, Joshua M. Jaffe, and Benjamin C. Jun. Differential power
771 analysis method and apparatus, December 2001.
- 772 [LCL10] Po-Chun Liu, Hsie-Chia Chang, and Chen-Yi Lee. A low overhead DPA
773 countermeasure circuit based on ring oscillators. *IEEE Trans. Circuits Syst.
774 II Express Briefs*, 57-II(7):546–550, 2010.
- 775 [Lun25] Fanny Lundberg. Exploiting ring oscillators to leak secret information. 2025.
- 776 [MGZS17] Jia-Wei Ma, Xuguang Guan, Tong Zhou, and Tao Sun. A new countermeasure
777 against side channel attack for HMAC-SM3 hardware. In Yajie Qin, Zhiliang
778 Hong, and Ting-Ao Tang, editors, *12th IEEE International Conference on
779 ASIC, ASICON 2017, Guiyang, China, October 25-28, 2017*, pages 327–330.
780 IEEE, 2017.

- [MOP07] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power analysis attacks: Revealing the secrets of smart cards*. Springer, 2007.
- [MOS11] Stefan Mangard, Elisabeth Oswald, and F-X Standaert. One for all—all for one: unifying standard differential power analysis attacks. *IET Information Security*, 5(2):100–110, 2011.
- [NPBH23] Hassan Nassar, Simon Pankner, Lars Bauer, and Jörg Henkel. Late breaking results: Configurable ring oscillators as a side-channel countermeasure. In *60th ACM/IEEE Design Automation Conference, DAC 2023, San Francisco, CA, USA, July 9-13, 2023*, pages 1–2. IEEE, 2023.
- [NYH13] Nianhao Zhu, Yujie Zhou, and Hongming Liu. Counteracting leakage power analysis attack using random ring oscillators. In *PROCEEDINGS OF 2013 International Conference on Sensor Network Security Technology and Privacy Communication System*, pages 74–77, Harbin, China, May 2013. IEEE.
- [PCT⁺15] Luis Parrilla, Encarnación Castillo, Elias Todorovich, Antonio García, Diego Pedro Morales, and Guillermo Botella. Improvements for the applicability of power-watermarking to embedded ip cores protection: E-coreipp. *Digital Signal Processing*, 44:110–122, 2015.
- [PG15] ProjectVault and Google Inc. orp: mselsoc AES crypto core (verilog rtl). https://github.com/ProjectVault/orp/tree/master/hardware/mselSoC/src/systems/geophyte/rtl/verilog/crypto_aes/rtl/verilog, 2015. GitHub repository, accessed April 16, 2026.
- [PGC⁺22] Luis Parrilla, Antonio García, Encarnación Castillo, Salvador Rodríguez-Bolívar, and Juan Antonio López-Villanueva. Time-and amplitude-controlled power noise generator against spa attacks for fpga-based iot devices. *Journal of Low Power Electronics and Applications*, 12(3):48, 2022.
- [PGX17] Satyajit Prabhakar Patne, Rajarshi Gupta, and Lu Xiao. Methods and systems for side channel analysis detection and protection, September 2017.
- [PHF] Thomas Plos, Michael Hutter, and Martin Feldhofer. Evaluation of Side-Channel Preprocessing Techniques on Cryptographic-Enabled HF and UHF RFID-Tag Prototypes.
- [Sch23] Patrick Schaumont. You Can Hide but You Can’t Verify: On Side-channel Countermeasure Verification. *Workshop on SSH-SoC, 2023*, 2023.
- [SEC] Ltd. Samsung Electronics Co. Security target lite of samsung s3nsn4v.
- [Sem] NXP Semiconductors. Sn220 series - secure element with crypto library.
- [Sha00] Adi Shamir. Protecting smart cards from passive power analysis with detached power supplies. In Çetin Kaya Koç and Christof Paar, editors, *CHES 2000*, volume 1965 of *LNCS*, pages 71–77. Springer, Berlin, Heidelberg, August 2000.
- [SMS20] Zeinab Seifoori, Seyedeh Sharareh Mirzargar, and Mirjana Stojilović. Closing leaks: Routing against crosstalk side-channel attacks. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 197–203, 2020.
- [SWL21] Dor Salomon, Amir Weiss, and Itamar Levi. Improved filtering techniques for single-and multi-trace side-channel analysis. *Cryptography*, 5(3):24, 2021.

- 825 [TAV02] Kris Tiri, Moonmoon Akmal, and Ingrid Verbauwhede. A dynamic and dif-
826 ferential cmos logic with signal independent power consumption to withstand
827 differential power analysis on smart cards. In *Proceedings of the 28th European*
828 *solid-state circuits conference*, pages 403–406. IEEE, 2002.
- 829 [TPZA25] Erica Tena-Sánchez, Francisco Eugenio Potestad-Ordóñez, Virginia Zúñiga-
830 González, and Antonio J Acosta. Low-cost full correlated-power-noise genera-
831 tor to counteract side-channel attacks. *Applied Sciences*, 15(6):3064, 2025.
- 832 [TV04] Kris Tiri and Ingrid Verbauwhede. A logic level design methodology for
833 a secure dpa resistant asic or fpga implementation. In *Proceedings Design,*
834 *Automation and Test in Europe Conference and Exhibition*, volume 1, pages
835 246–251. IEEE, 2004.
- 836 [YKS⁺23] Yuan Yao, Pantea Kiaei, Richa Singh, Shahin Tajik, and Patrick Schaumont.
837 Programmable ro (pro): A multipurpose countermeasure against side-channel
838 and fault injection attack. In *Security of FPGA-Accelerated Cloud Computing*
839 *Environments*, pages 297–325. Springer, 2023.